

Customer Shopping Behavior Analysis

Python · MySQL · Power BI

1. Project Overview

This project analyzes customer shopping behavior using transactional data from **3,900 purchases** across various product categories. The goal is to uncover insights into spending patterns, customer segments, product preferences, and subscription behavior to guide strategic business decisions.

2. Dataset Summary

Attribute	Detail
Rows	3,900
Columns	18
Missing Data	37 values in Review Rating column

Key Features

- **Customer Demographics:** Age, Gender, Location, Subscription Status
- **Purchase Details:** Item Purchased, Category, Purchase Amount, Season, Size, Color
- **Shopping Behavior:** Discount Applied, Previous Purchases, Frequency of Purchases, Review Rating, Shipping Type

3. Exploratory Data Analysis using Python

Data preparation and cleaning was performed in Python. Below are the key steps taken, along with the relevant code.

Missing Data Handling

Checked for null values and imputed missing values in the Review Rating column using the **median rating of each product category** to reduce outlier impact.

```
df["Review Rating"] = df.groupby("Category")["Review Rating"]
                        .transform(lambda x: x.fillna(x.median()))
```

Column Standardization

Renamed all columns to **snake_case** for consistency. Renamed purchase_amount_(usd) to purchase_amount.

```
df.columns = df.columns.str.lower().str.replace(" ", "_")

df = df.rename(columns={"purchase_amount_(usd)": "purchase_amount"})
```

Feature Engineering

Created an **age_group** column by binning customer ages into four segments, and a **purchase_frequency_days** column by mapping purchase intervals to numeric day values.

```
# Age grouping

labels = ['Young Adult', 'Adult', 'Middle-aged', 'Senior']

df['age_group'] = pd.qcut(df['age'], q=4, labels=labels)

# Purchase frequency in days

frequency_mapping = {

'Weekly': 7, 'Fortnightly': 14, 'Bi-Weekly': 14,

'Monthly': 30, 'Quarterly': 90,

'Every 3 Months': 90, 'Annually': 365

}

df['purchase_frequency_days'] = df['frequency_of_purchases'].map(frequency_mapping)
```

Data Consistency Check

Verified that **discount_applied** and **promo_code_used** were identical columns; dropped **promo_code_used** to eliminate redundancy.

```
(df['discount_applied'] == df['promo_code_used']).all() # True

df = df.drop('promo_code_used', axis=1)
```

Database Integration

Connected the cleaned DataFrame to a **MySQL** database using **SQLAlchemy** for SQL-based analysis.

```
from sqlalchemy import create_engine

engine = create_engine(

"mysql+pymysql://root:****@localhost:3306/customer_behavior_db"

)

df.to_sql(name="customers", con=engine, if_exists="replace", index=False)
```

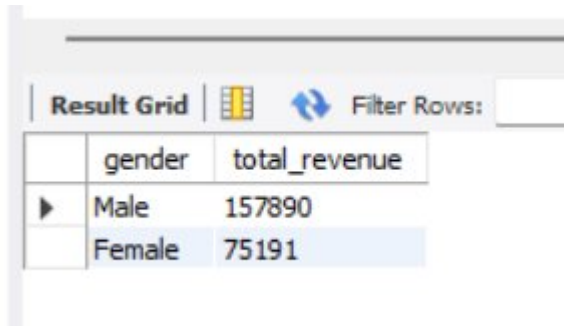
4. Data Analysis using SQL (Business Transactions)

Structured analysis was performed in MySQL to answer key business questions.

1. Revenue by Gender

Compared total revenue generated by male vs. female customers.

```
SELECT gender, SUM(purchase_amount) AS total_revenue  
FROM customers  
GROUP BY gender;
```



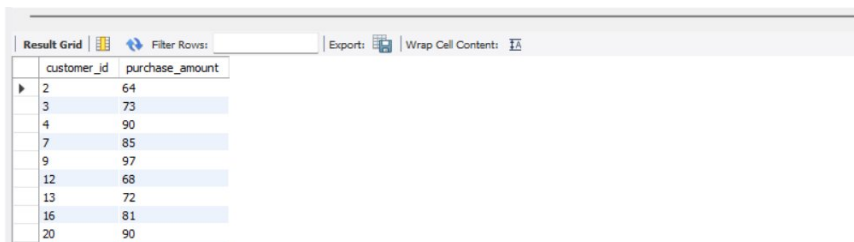
The screenshot shows a 'Result Grid' interface with a 'Filter Rows' button. The table has two columns: 'gender' and 'total_revenue'. The data is as follows:

	gender	total_revenue
▶	Male	157890
	Female	75191

2. High-Spending Discount Users

Identified customers who used discounts but still spent above the average purchase amount.

```
SELECT customer_id, purchase_amount  
FROM customers  
WHERE discount_applied = 'Yes'  
AND purchase_amount >= (SELECT AVG(purchase_amount) FROM customers);
```



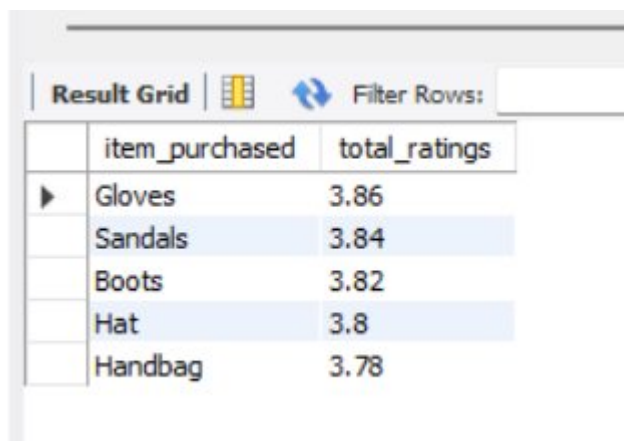
The screenshot shows a 'Result Grid' interface with a 'Filter Rows' button, an 'Export' button, and a 'Wrap Cell Content' checkbox. The table has two columns: 'customer_id' and 'purchase_amount'. The data is as follows:

	customer_id	purchase_amount
▶	2	64
	3	73
	4	90
	7	85
	9	97
	12	68
	13	72
	16	81
	20	90

3. Top 5 Products by Rating

Found products with the highest average review ratings.

```
SELECT item_purchased,  
ROUND(AVG(review_rating), 2) AS total_ratings  
FROM customers  
GROUP BY item_purchased  
ORDER BY total_ratings DESC  
LIMIT 5;
```



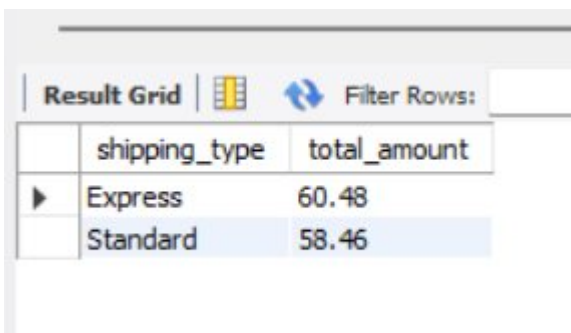
The screenshot shows a 'Result Grid' interface with a 'Filter Rows' button. The table displays the top 5 products by their average review rating, sorted in descending order. The columns are 'item_purchased' and 'total_ratings'.

	item_purchased	total_ratings
▶	Gloves	3.86
	Sandals	3.84
	Boots	3.82
	Hat	3.8
	Handbag	3.78

4. Shipping Type Comparison

Compared average purchase amounts between Standard and Express shipping.

```
SELECT shipping_type,  
ROUND(AVG(purchase_amount), 2) AS total_amount  
FROM customers  
WHERE shipping_type IN ('Standard', 'Express')  
GROUP BY shipping_type;
```



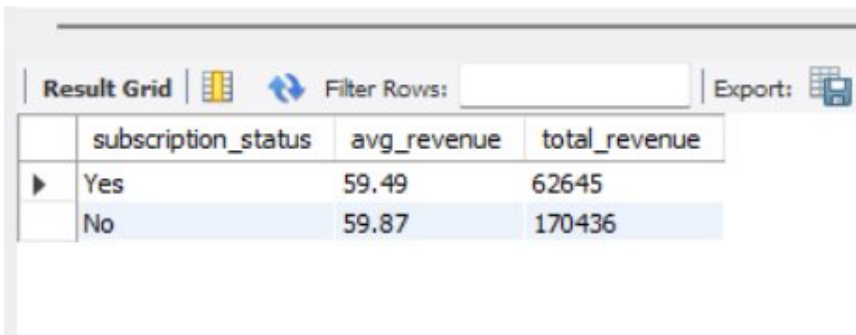
The screenshot shows a 'Result Grid' interface with a 'Filter Rows' button. The table displays the average purchase amount for Standard and Express shipping types. The columns are 'shipping_type' and 'total_amount'.

	shipping_type	total_amount
▶	Express	60.48
	Standard	58.46

5. Subscribers vs. Non-Subscribers

Compared average spend and total revenue across subscription status.

```
SELECT subscription_status,  
ROUND(AVG(purchase_amount), 2) AS avg_revenue,  
SUM(purchase_amount) AS total_revenue  
FROM customers  
GROUP BY subscription_status  
ORDER BY avg_revenue, total_revenue DESC;
```



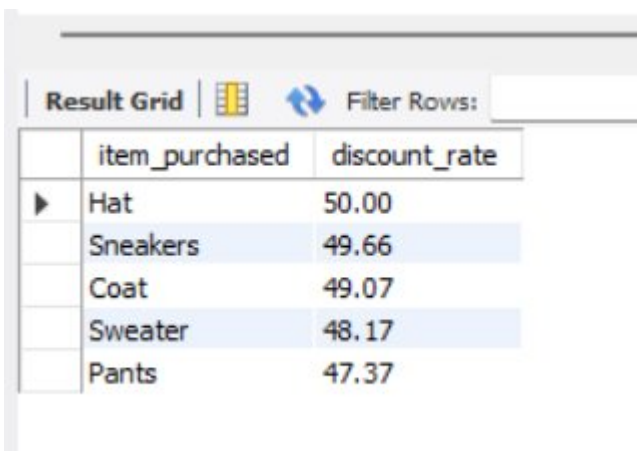
The screenshot shows a database query result grid with the following data:

	subscription_status	avg_revenue	total_revenue
▶	Yes	59.49	62645
	No	59.87	170436

6. Discount-Dependent Products

Identified 5 products with the highest percentage of discounted purchases.

```
SELECT item_purchased,  
ROUND(100.0 * SUM(CASE WHEN discount_applied = 'Yes'  
THEN 1 ELSE 0 END) / COUNT(*), 2) AS discount_rate  
FROM customers  
GROUP BY item_purchased  
ORDER BY discount_rate DESC  
LIMIT 5;
```



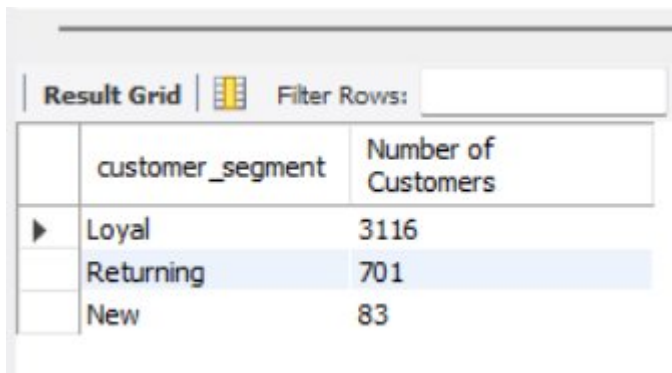
The screenshot shows a database query result grid with the following data:

	item_purchased	discount_rate
▶	Hat	50.00
	Sneakers	49.66
	Coat	49.07
	Sweater	48.17
	Pants	47.37

7. Customer Segmentation

Classified customers into New, Returning, and Loyal segments based on purchase history.

```
WITH customer_type AS (  
  SELECT customer_id, previous_purchases,  
  CASE WHEN previous_purchases = 1 THEN 'New'  
  WHEN previous_purchases BETWEEN 2 AND 10 THEN 'Returning'  
  ELSE 'Loyal' END AS customer_segment  
  FROM customers  
)  
SELECT customer_segment, COUNT(*) AS 'Number of Customers'  
FROM customer_type  
GROUP BY customer_segment;
```



The screenshot shows a 'Result Grid' window with a 'Filter Rows' input field. The grid contains three columns: an index column, 'customer_segment', and 'Number of Customers'. The data is as follows:

	customer_segment	Number of Customers
▶	Loyal	3116
	Returning	701
	New	83

8. Top 3 Products per Category

Listed the most purchased products within each category.

```
WITH items_count AS (  
  SELECT category, item_purchased, COUNT(*) AS total_orders,  
  ROW_NUMBER() OVER (  
    PARTITION BY category  
    ORDER BY COUNT(customer_id) DESC) AS rk  
  FROM customers  
  GROUP BY category, item_purchased  
)  
SELECT category, item_purchased, total_orders  
FROM items_count WHERE rk <= 3;
```

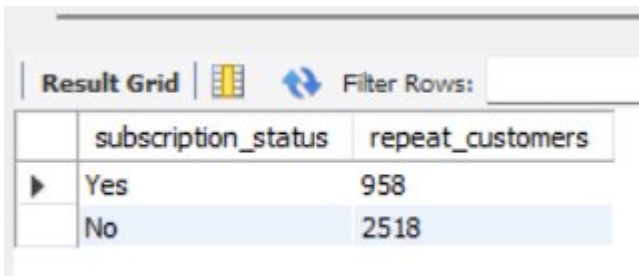
Result Grid			
		Filter Rows:	
Export:			
	category	item_purchased	total_orders
▶	Accessories	Jewelry	171
	Accessories	Sunglasses	161
	Accessories	Belt	161
	Clothing	Blouse	171
	Clothing	Pants	171
	Clothing	Shirt	169
	Footwear	Sandals	160
	Footwear	Shoes	150
	Footwear	Sneakers	145
	Outerwear	Jacket	163
	Outerwear	Coat	161

Result 11 ×

9. Repeat Buyers & Subscriptions

Checked whether customers with more than 5 previous purchases are more likely to subscribe.

```
SELECT subscription_status,  
COUNT(customer_id) AS repeat_buyers  
FROM customers  
WHERE previous_purchases > 5  
GROUP BY subscription_status;
```



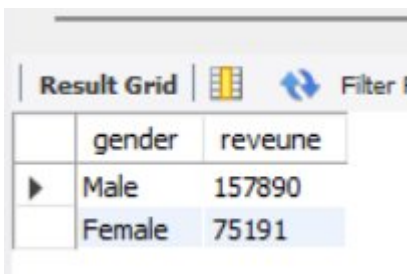
The screenshot shows a 'Result Grid' with two columns: 'subscription_status' and 'repeat_customers'. The first row shows 'Yes' with a count of 958. The second row shows 'No' with a count of 2518.

	subscription_status	repeat_customers
▶	Yes	958
	No	2518

10. Revenue by Age Group

Calculated total revenue contribution of each age group.

```
SELECT age_group, SUM(purchase_amount) AS revenue  
FROM customers  
GROUP BY age_group  
ORDER BY revenue DESC;
```



The screenshot shows a 'Result Grid' with two columns: 'gender' and 'revenue'. The first row shows 'Male' with a revenue of 157890. The second row shows 'Female' with a revenue of 75191.

	gender	revenue
▶	Male	157890
	Female	75191

5. Dashboard in Power BI

An interactive dashboard was built in **Power BI** to present insights visually.

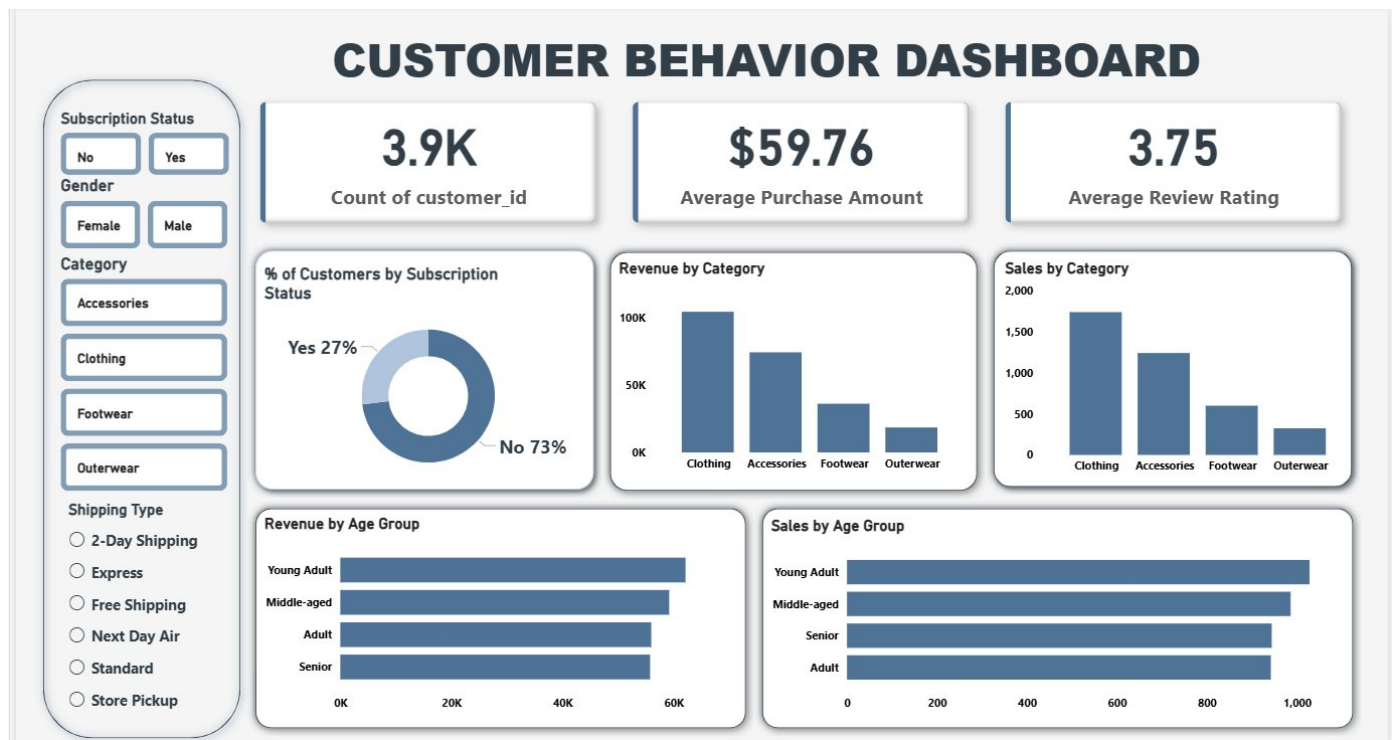


Figure 1 — Customer Behavior Dashboard (Power BI)

6. Business Recommendations

- **Boost Subscriptions** — Promote exclusive benefits for subscribers.
- **Customer Loyalty Programs** — Reward repeat buyers to move them into the Loyal segment.
- **Review Discount Policy** — Balance sales boosts with margin control.
- **Product Positioning** — Highlight top-rated and best-selling products in campaigns.
- **Targeted Marketing** — Focus efforts on high-revenue age groups and express-shipping users.

End of Report